Experiment No. 7
Creation and Deployment of Smart Contract and Transactions using Solidity and Remix IDE

Aim

To create and deploy a smart contract using Solidity and perform transactions on a local Ethereum blockchain using the Truffle Suite and Ganache.

Theory

Local blockchains. Developing DApps on the Ethereum mainnet or public testnets involves transaction latency, rate limits and gas expenses. A local blockchain simulates the Ethereum Virtual Machine (EVM) on a personal workstation, with instant block mining, pre-funded virtual accounts and deterministic state, so developers can iterate without risking real assets.

Truffle Suite. A development environment, testing framework and asset pipeline for EVM-based blockchains.

- **Compilation** – converts Solidity (`.sol`) into EVM bytecode and ABI.
- **Migrations** – scriptable deployment pipeline that tracks earlier deployments on-chain.
- **Network management** – local, private, testnet and mainnet configured in `truffle-config.js`.
- **Console and scripts** – Node.js runtime with Web3 (`truffle console`, `truffle exec`).

Project structure: `contracts/` (Solidity sources, `Migrations.sol`), `migrations/` (numbered deployment scripts), `test/` (test suites), `build/contracts/` (JSON artifacts: bytecode, metadata, AST, ABI) and `truffle-config.js` (RPC host, port, gas, network ID, compiler version).

Ganache. Local EVM simulator of the Truffle Suite; starts a private Ethereum node with a JSON-RPC interface (typically `127.0.0.1:7545` or `8545`).

- 10 deterministic accounts from a 12-word mnemonic, each with 100 test ETH.
- Automine – a block is mined immediately for every valid transaction.
- Private keys are exposed, so accounts can be imported into MetaMask.
- Gas price can be near zero, so testing does not drain balances.

Bytecode and ABI. Bytecode is the hexadecimal opcode sequence executed by the EVM. The ABI is a JSON description of the public functions, arguments, return values and events; it lets scripts (Web3.js, Ethers.js) encode calls and decode results.

Migrations. `Migrations.sol` stores a single variable, `last_completed_migration`. On `truffle migrate`, only scripts numbered higher than it are run, which prevents duplicate deployments. The `deployer` object handles deployment order, library linking and constructor arguments.

Calls vs. transactions.

- **Call** (`view`/`pure`, e.g. `get()`) – read-only, executed locally on the node, needs no block and no gas.
- **Transaction** (e.g. `set(uint x)`) – changes state, is signed with the sender's private key, mined into a block, gets a transaction hash and consumes gas.

Requirements

- Windows computer with Node.js and npm.
- Truffle Suite (`truffle`) and Ganache 2.7.1.
- Solidity compiler 0.8.20 (solc-js) – `SimpleStorage.sol` contract.

Procedure

Step 1: Install Ganache

Install Ganache and start a local blockchain workspace (RPC server at `127.0.0.1:7545`).

Step 2: Compile the contracts

In the project folder `Exp_7_blockchain`, run `truffle compile` to compile `Migrations.sol` and `SimpleStorage.sol`; artifacts are written to `build/contracts`.

Step 3: Migrate

Run `truffle migrate`. `1_initial_migration.js` deploys `Migrations` and `2_deploy_contracts.js` deploys `SimpleStorage` to the local network.

Step 4: Interact with the contract

Run `truffle exec scripts/interact.js` to send the transaction `set(100)` and read the value back with `get()`.

Step 5: Verify in Ganache

Open the Transactions and Blocks tabs of Ganache to confirm the contract creations, the calls and the mined blocks.

Commands Executed

```
npx truffle compile
npx truffle migrate
npx truffle exec scripts/interact.js
```

Output

Ganache installation

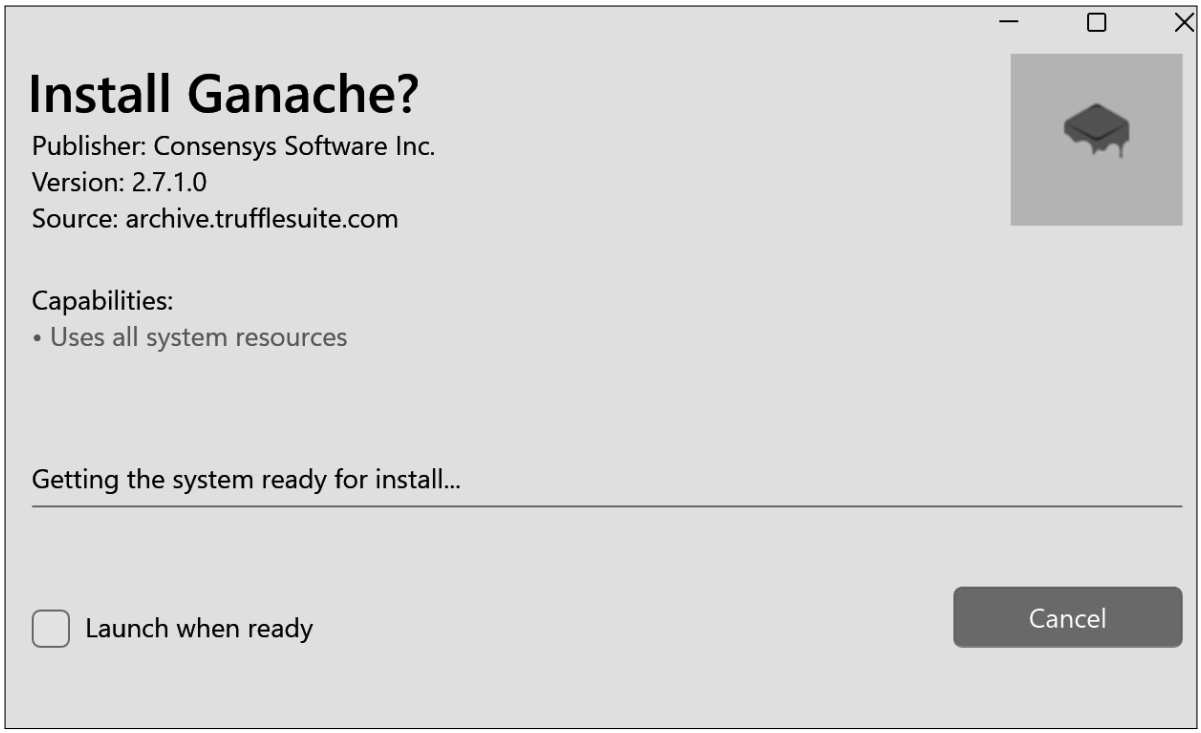


Figure 1: Installing Ganache

Compilation

```
PS C:\Users\heram\Desktop\Exp_7_blockchain> npx truffle compile
>>

Compiling your contracts...
===========================
> Compiling .\contracts\Migrations.sol
> Compiling .\contracts\SimpleStorage.sol
> Artifacts written to C:\Users\heram\Desktop\Exp_7_blockchain\build\contracts
> Compiled successfully using:
   - solc: 0.8.20+commit.a1b79de6.Emscripten.clang
```

Figure 2: `truffle compile` – contracts compiled successfully

Migration and deployment

```
Compiling your contracts...
===========================
> Compiling .\contracts\Migrations.sol
> Compiling .\contracts\SimpleStorage.sol
> Artifacts written to C:\Users\heram\Desktop\Exp_7_blockchain\build\contracts
> Compiled successfully using:
   - solc: 0.8.20+commit.a1b79de6.Emscripten.clang


Starting migrations...
======================
> Network name:    'development'
> Network id:      5777
> Block gas limit: 6721975 (0x6691b7)


1_initial_migration.js
======================

   Replacing 'Migrations'
   ----------------------
   > transaction hash:    0x7f8bd17c70dd2bded2987e5042b4dc55af30c5bf71d60320312e8eeb2467a335
   > Blocks: 0            Seconds: 0
   > contract address:    0x5c6ec88fF9A3A9677F1e01B4f894F87Fb84C4ff6
   > block number:        6
   > block timestamp:     1791389013
   > account:             0xc8637eBF4F8e148B2CAe4Bf5D0C844D60836c2C7
   > balance:             99.99764469459288602
   > gas used:            250154 (0x3d12a)
   > gas price:           2.958288746 gwei
   > value sent:          0 ETH
```

Figure 3: `truffle migrate` – `Migrations` deployed (network id 5777)

```
   > gas price:           2.958288746 gwei
   > value sent:          0 ETH
   > total cost:          0.000740027762966884 ETH

   > Saving migration to chain.
   > Saving artifacts
   -------------------------------------
   > Total cost:     0.000740027762966884 ETH


2_deploy_contracts.js
=====================

   Replacing 'SimpleStorage'
   > Blocks: 0            Seconds: 0
   > contract address:    0xe25074697d309198a22dB4E8256893439f56C4fE
   > block number:        8
   > block timestamp:     1791389013
   > account:             0xc8637eBF4F8e148B2CAe4Bf5D0C844D60836c2C7
   > balance:             99.997152528072755338
   > gas used:            125653 (0x1ead5)
   > gas price:           2.855300109 gwei
   > value sent:          0 ETH
   > total cost:          0.000358777024596177 ETH

   > Saving migration to chain.
   > Saving artifacts
   -------------------------------------
   > Total cost:     0.000358777024596177 ETH
```

Figure 4: `2_deploy_contracts.js` – `SimpleStorage` deployed

```
Summary
=======
> Total deployments:   2
> Final cost:          0.001098804787563061 ETH


PS C:\Users\heram\Desktop\Exp_7_blockchain>
```

Figure 5: Deployment summary – 2 deployments

Interacting with the contract

```
PS C:\Users\heram\Desktop\Exp_7_blockchain> npx truffle exec scripts/interact.js
>>
This version of µWS is not compatible with your Node.js build:

Error: Cannot find module '../binaries/uws_win32_x64_137.node'
Require stack:
- C:\Users\heram\Desktop\Exp_7_blockchain\node_modules\ganache\node_modules\@trufflesuite\uws
-js-unofficial\src\uws.js
- C:\Users\heram\Desktop\Exp_7_blockchain\node_modules\ganache\dist\node\core.js
- C:\Users\heram\Desktop\Exp_7_blockchain\node_modules\truffle\build\exec.bundled.js
- C:\Users\heram\Desktop\Exp_7_blockchain\node_modules\original-require\index.js
- C:\Users\heram\Desktop\Exp_7_blockchain\node_modules\truffle\build\cli.bundled.js
Falling back to a NodeJS implementation; performance may be degraded.


Using network 'development'.

-------------------------------------------------
Connected to deployed SimpleStorage at: 0xe25074697d309198a22dB4E8256893439f56C4fE
Executing transaction: set(100)...
Transaction Hash: 0xa9a2e71951a3b49445534ec1f11c6c3b9878de85df8dac95791bbf23d445e6a2
Block Number: 10
Gas Used: 43702
Calling get()...
Current Stored Value: 100
-------------------------------------------------
PS C:\Users\heram\Desktop\Exp_7_blockchain>
```

Figure 6: `interact.js` – `set(100)` transaction and `get()` returns 100

Ganache

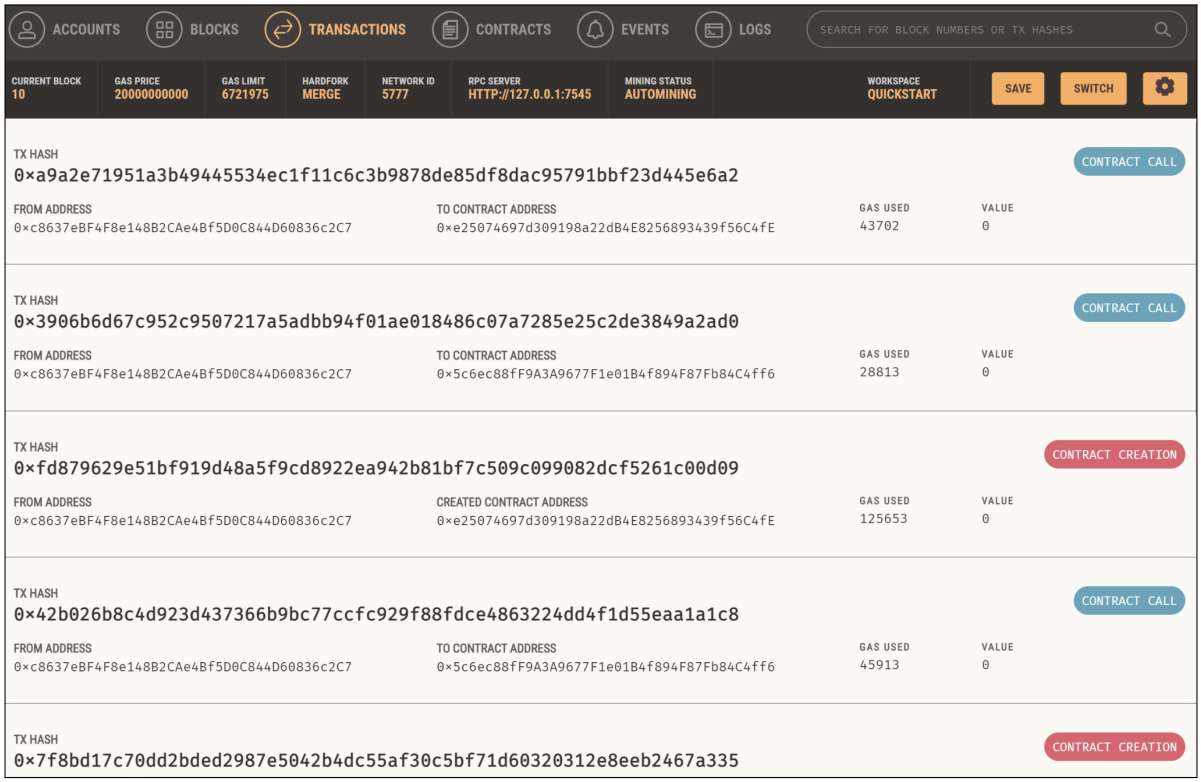


Figure 7: Ganache Transactions tab – contract creations and calls

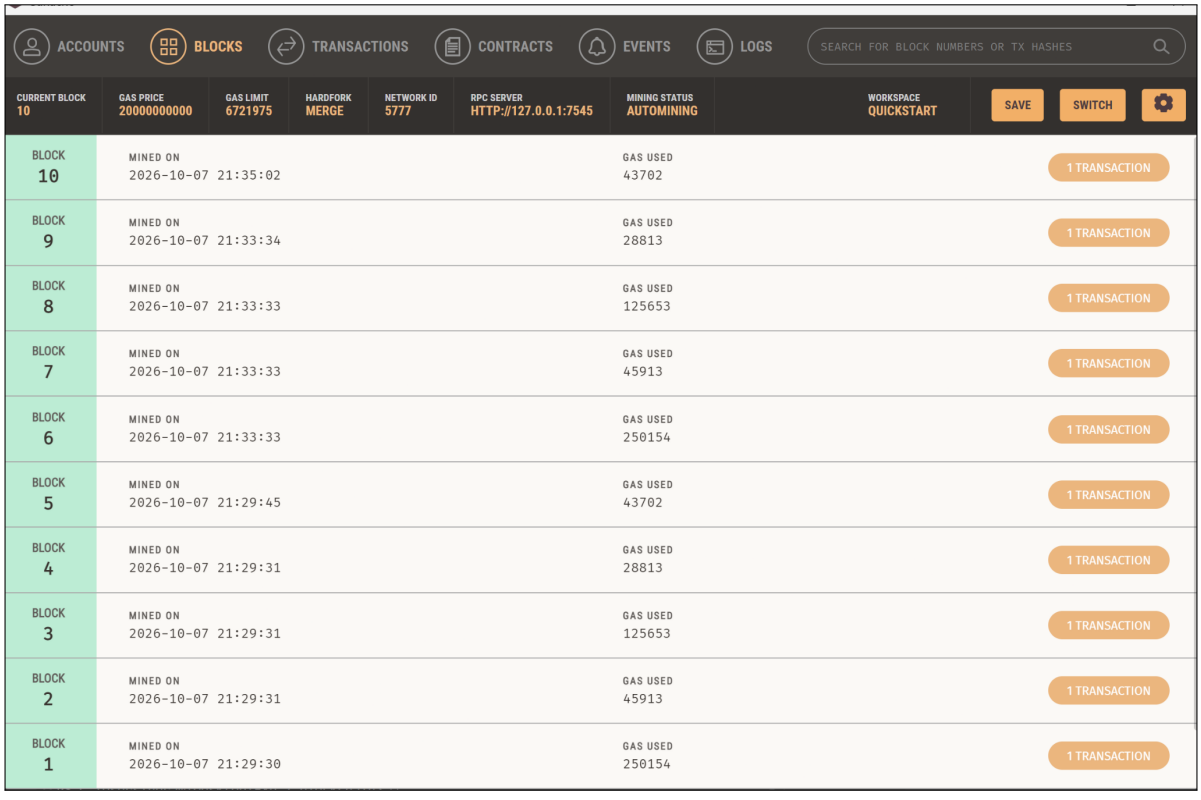


Figure 8: Ganache Blocks tab – mined blocks

Conclusion

The experiment was successfully executed by establishing a local Ethereum development pipeline using the Truffle Suite and Ganache. The smart contract was written, compiled into EVM bytecode and ABI artifacts, and migrated onto the local private blockchain running at `127.0.0.1:7545`. Through automated script execution, state-changing write transactions and view-based read calls were verified, confirming state persistence and block creation on the local ledger without incurring real gas fees. This gave a complete foundational understanding of local node configuration, automated migration scripts and decentralized application interaction workflows.

Experiment completed successfully.